

Meridian — Project Proposal

A shared workspace that keeps working when the internet does not.

Author: Saeed Hussain

Written: 17 September 2026

Language: Pure JavaScript (ES2023+) — no TypeScript

Stack: Next.js · Electron · WebRTC · IndexedDB · SQLite

Time needed: 10 to 12 weeks

1. What is Meridian?

Meridian is an app where a team writes documents together.

Two things make it different from Google Docs or Notion:

- 1. It works offline.** You can keep typing with no internet. Nothing is blocked and nothing is lost.
- 2. It does not need a server.** Computers can send changes straight to each other. The server is only a helper, not the boss.

When everyone comes back online, all the changes are merged. Every person ends up with exactly the same document. Nobody has to fix anything by hand.

2. The problem

Most team apps today have the same weakness. They need the internet.

- Your internet drops for ten minutes, and the app stops working.
- You open your laptop on a bus with no signal, and you cannot edit.
- Two people edit the same line, and one person's work is quietly lost.
- The company that runs the server can read everything you write.

For a user in a place where the internet is slow or drops often, this is a daily problem, not a rare one.

3. Who it is for

- Small teams who write notes, plans and documents together.
 - People with slow or unreliable internet.
 - Teams who do not want their private text sitting on someone else's server.
-

4. What Meridian will do

Version 1 features:

- Write and edit documents together, live.
 - See other people's cursors and names while they type.
 - Keep working with no internet. Changes are saved on your own device.
 - Merge everything automatically when the connection returns.
 - Send changes directly between computers, not through a server.
 - Encrypt the text, so the helper server cannot read it.
 - Look at the history of a document and go back to any earlier point.
 - Run in a web browser and as a Windows desktop app.
-

5. The hard part, explained simply

Here is the real problem this project solves.

Imagine a document with one line:

> `Hello world`

Two people are offline at the same time.

- **Person A** adds `beautiful` in the middle. They see `Hello beautiful world`.
- **Person B** adds `big` in the same place. They see `Hello big world`.

Now both come back online. What should the document say?

A normal app would pick one and throw the other away. Someone loses their work.

Meridian must do better. Both edits must survive, and — this is the important part — **both people must see the same final result**. Not similar. The same. Letter for letter. Even if the changes arrive in a different order on each computer.

How it is solved

The answer is a method called a **CRDT**. The short name stands for *Conflict-free Replicated Data Type*. That sounds complex, but the idea is simple:

> Give every single letter its own permanent ID. Then write merge rules that always > give the same answer, no matter what order the changes arrive in.

Because the rules never depend on timing or on who is "first", every copy of the document ends up identical on its own. No server has to act as a judge.

Why this is worth doing

Almost every developer who builds a feature like this installs a ready-made library (Yjs or Automerge) and moves on.

I am going to write the merge algorithm myself.

That is the whole point of this project. Anyone can install a library. Very few people can explain how the merging actually works, and fewer still can prove that their version is correct. That proof is what makes this project worth the time.

6. How it works

The system has four parts.

Part 1 — The core (plain JavaScript)

This is the brain. It holds the document, applies changes and does the merging. It is a plain JavaScript package. It knows nothing about React, Next.js or the browser, so it can run anywhere and it is easy to test.

Part 2 — Storage

Every change is written to your own device first.

- In the browser: IndexedDB.
- In the desktop app: SQLite.

Because of this, your work is safe even if you close the laptop with no internet.

Part 3 — Networking

Computers connect to each other using **WebRTC data channels**. This is the same technology used for video calls, but here it carries text changes instead of video.

A small server helps two computers find each other at the start. After that, the data goes directly between them. The server never sees the document.

Part 4 — The interface

A Next.js app gives the editor, the cursors, the history view and the connection status. The desktop version is the same interface wrapped in Electron.

The flow, step by step

1. You type a letter.
2. The core turns it into a change with a unique ID.
3. The change is saved on your device immediately.
4. The change is encrypted and sent to connected friends.
5. Their core applies it using the merge rules.

- 6. Their screen updates.
- 7. If you were offline, steps 4 to 6 simply happen later. Nothing is lost.

7. What I will build myself

This matters, so it is written down clearly.

PART	BUILT BY ME	USING A LIBRARY
Merge algorithm (CRDT)	Yes	No
Document history and snapshots	Yes	No
Sync protocol between devices	Yes	No
Storage layer	Yes	No
Encryption	The design	Browser crypto for the maths
WebRTC connection	The logic	Browser WebRTC API
Interface	Yes	Next.js and React

The rule is simple: **the thinking parts are mine**. For low-level maths, such as encryption, using the browser's built-in tools is correct and safe. Writing your own encryption maths is a bad idea, and doing so would be a mistake, not a strength.

8. Plan of work

The work is split into six stages over 10 to 12 weeks.

STAGE	WEEKS	GOAL
1. Setup	1	Project structure, tests and CI running.
2. The core	2–4	Merging works. Proved by tests.
3. Saving	5	Work survives a refresh and a crash.
4. Networking	6–7	Two browsers sync directly, live.
5. The app	8–9	A real editor with cursors and offline mode.
6. Finish	10–12	Encryption, desktop app, benchmarks, release.

A full day-by-day plan is in the companion document, [WORKPLAN.pdf](#).

9. How I will know it works

These are the tests the project must pass. They are not opinions, so they cannot be argued with.

- 1. The four-window test.** Open the same document in four browser windows. Turn off the network on two of them. Type wildly in all four. Reconnect. All four windows must show exactly the same text.
 - 2. The random test.** A script creates thousands of random edits, applies them in every possible order, and checks that every copy matches. This is the real proof.
 - 3. The crash test.** Kill the app in the middle of typing. Reopen it. No work is lost.
 - 4. The speed test.** A document with 100,000 letters must still feel instant while typing.
 - 5. The offline test.** Work for one hour with no internet. Reconnect. Everything merges with no errors and no duplicates.
-

10. Risks, and what I will do about them

RISK	HOW LIKELY	WHAT I WILL DO
The merge algorithm has a rare bug	High	Use random testing from week 2, not at the end. Bugs found early are cheap.
The document gets slow when large	Medium	Measure speed every week. Fix the data structure as soon as it slows down.
WebRTC fails behind strict networks	Medium	Add a fallback relay server. Report honestly when direct connection is not possible.
The project grows too big to finish	High	Keep version 1 small. The list in section 11 stays out, no matter how tempting.
I get stuck on one hard piece	Medium	Give any single problem three days maximum, then ship a simpler version and move on.

The biggest risk is not technical. It is starting something hard and not finishing it. An unfinished project is worth nothing, so the plan is deliberately small.

11. Not in version 1

Writing this list down is what keeps the project finishable.

- No comments or replies.
- No mobile apps.
- No file uploads or images.
- No user accounts with passwords. Sharing is by link and key.
- No rich text at first. Plain text first, formatting later.
- No permissions or roles.

Each of these can come later. None of them is needed to prove the hard part works.

12. What will exist at the end

- A working web app anyone can open and try.
 - A signed Windows desktop app.
 - A JavaScript package holding the merge algorithm, with its own documentation.
 - A test suite that proves the merging is correct.
 - Speed numbers, published honestly, including where it is slow.
 - A short recording of the four-window test, because it is the demo that explains everything in ten seconds.
 - A written article explaining how the merge algorithm works.
-

13. Why this project is worth the time

Meridian brings together three skills that rarely appear in one person:

- 1. Offline-first software** — already proven at work on Taajir's point-of-sale.
- 2. Peer-to-peer connections** — already proven on Grove with WebRTC video.
- 3. Distributed algorithms written from scratch** — the new skill this project adds.

The first two are on my CV today. The third is the rare one. Building it turns "I have used offline sync" into "I wrote the algorithm and proved it correct."

That is a much harder sentence for another developer to match.